



Flyweight Pattern

1 Mục tiêu bài học

Sau bài học này, sinh viên có thể:

- Hiểu mục đích và ý tưởng cốt lõi của Flyweight Pattern
- Phân biệt Intrinsic State và Extrinsic State
- Nhận diện vấn đề dư thừa bộ nhớ trong hệ thống lớn
- Áp dụng Flyweight vào Order Management System (OMS)
- Triển khai Flyweight Pattern bằng C#

2 Vấn đề

Trong hệ thống lớn như Order Management System, ta có thể có:
Hàng triệu OrderItem | Hàng nghìn Product giống nhau | Nhiều đơn hàng cùng sản phẩm.

Ví dụ sai:

```
public class Product { string Name; string Category; decimal Price; }
```

Nếu mỗi OrderItem tạo một Product mới (Order 1, 2, 3 → Product A):

 Vấn đề:

- Lãng phí bộ nhớ
- Tốn tài nguyên
- Hiệu suất kém khi hệ thống scale

3 Giải pháp & **4** Khái niệm quan trọng

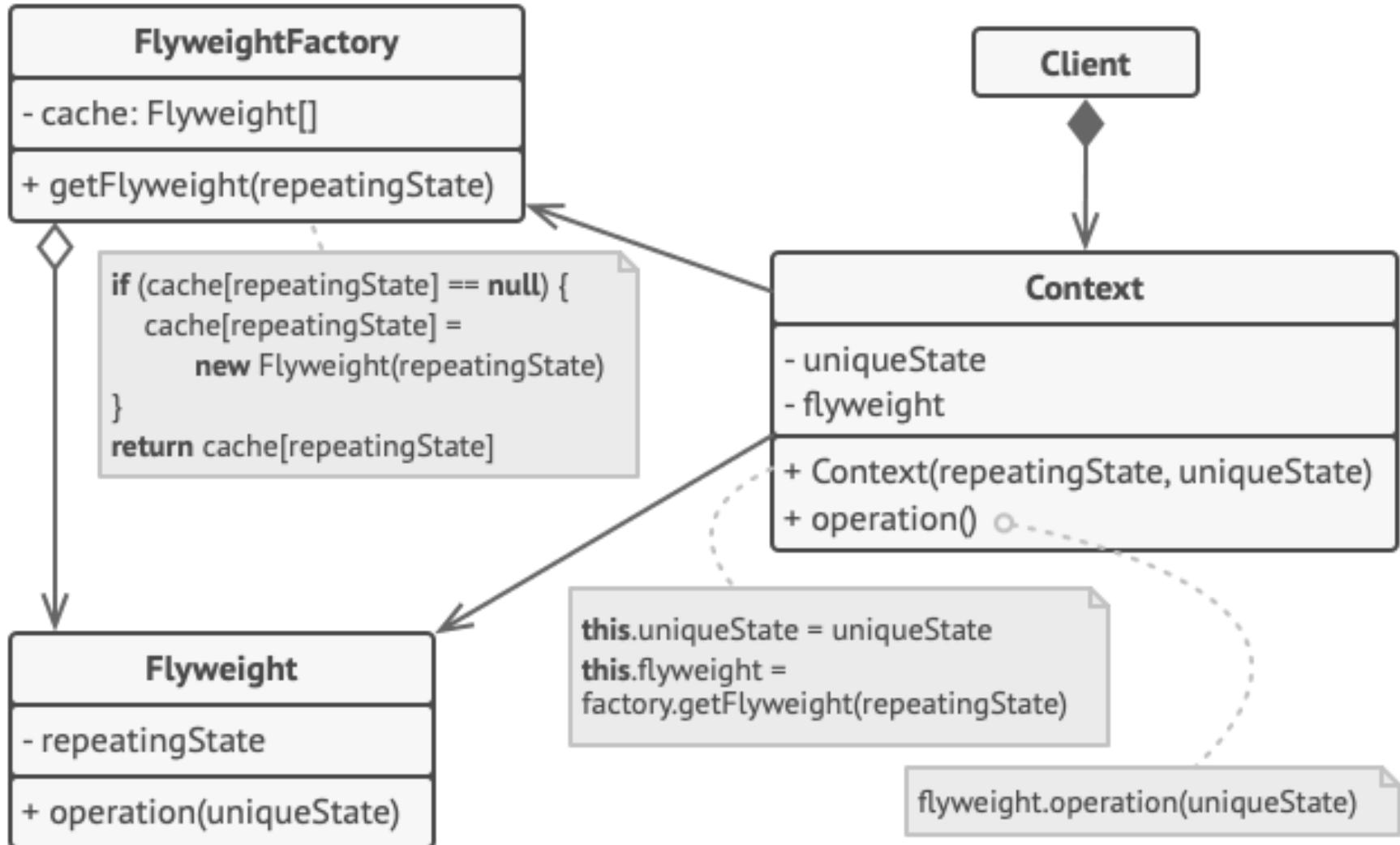
3 Giải pháp:

- ◆ Flyweight giúp chia sẻ các đối tượng giống nhau để giảm bộ nhớ.
- ◆ Tách trạng thái dùng chung (Intrinsic State)
- ◆ Tách trạng thái riêng (Extrinsic State)
- ◆ Dùng Flyweight Factory để quản lý object dùng chung

4 Khái niệm quan trọng

- ◆ Intrinsic State (nội tại – dùng chung): Tên sản phẩm, Giá, Category.
- ◆ Extrinsic State (bên ngoài – riêng instance): Số lượng, OrderId, Discount.

3 Structure of Flyweigh pattern



Ví dụ 1: Product Flyweight trong OMS (Phần 1)

Bước 1: Flyweight Interface

```
public interface IProductFlyweight { void Display(string orderId, int quantity); }
```

Bước 2: Concrete Flyweight

```
public class ProductFlyweight : IProductFlyweight {  
    private string _name; private string _category; private decimal _price;  
    public ProductFlyweight(string n, string c, decimal p) { _name=n;  
_category=c; _price=p; }  
    public void Display(string orderId, int quantity) {  
        Console.WriteLine($"Order: {orderId}, Product: {_name}, Quantity:  
{quantity}");  
    }  
}
```



Ví dụ 1: Product Flyweight trong OMS (Phần 2)



Bước 3: Flyweight Factory

```
public class ProductFactory {  
    private Dictionary<string, IProductFlyweight> _products = new();  
    public IProductFlyweight GetProduct(string name, string cat, decimal  
price) {  
        if (!_products.ContainsKey(name)) _products[name] = new  
ProductFlyweight(name, cat, price);  
        return _products[name];  
    }  
}
```



Sử dụng

```
var p1 = factory.GetProduct("Laptop", "Electronics", 1500);  
var p2 = factory.GetProduct("Laptop", "Electronics", 1500);  
Console.WriteLine(Object.ReferenceEquals(p1, p2)); //  
True
```



Ví dụ 2: Discount Flyweight

Giả sử hệ thống có nhiều loại giảm giá: 10%, 20%, Black Friday.



Concrete Flyweight

```
public class PercentageDiscount : IDiscountFlyweight {  
    private decimal _percentage;  
    public PercentageDiscount(decimal p) { _percentage = p; }  
    public decimal ApplyDiscount(decimal amount) => amount - (amount *  
_percentage);  
}
```



Factory

```
public class DiscountFactory {  
    private Dictionary<string, IDiscountFlyweight> _discounts = new();  
    public IDiscountFlyweight GetDiscount(string key, decimal pct) {  
        if (!_discounts.ContainsKey(key)) _discounts[key] = new  
PercentageDiscount(pct);  
        return _discounts[key];  
    }  
}
```




So sánh Trước & Sau | Flyweight vs Singleton

Không dùng Flyweight	Dùng Flyweight
Nhiều object giống	Chia sẻ object
Tốn RAM	Tiết kiệm RAM
Khó scale	Scale tốt

Flyweight	Singleton
Nhiều instance chia sẻ	Chỉ một instance
Dùng Factory quản lý	Instance toàn cục
Tối ưu bộ nhớ	Kiểm soát truy cập



Khi nào nên dùng & Bài tập thực hành



Khi nào nên dùng Flyweight?

- ✓ Object cực lớn | ✓ Dữ liệu giống nhau | ✓ RAM là vấn đề | ✓ Tách được intrinsic/extrinsic.
- ✗ Không nên dùng nếu object ít hoặc đơn giản.



Bài tập thực hành

Bài 1: Thiết kế Flyweight cho ShippingMethod (Standard, Express, SameDay).

Bài 2: Giả lập 1 triệu OrderItem, so sánh memory usage.

Bài 3: Kết hợp Flyweight + Factory và Flyweight + Proxy.



Kết luận

Flyweight Pattern giúp: Giảm tiêu thụ bộ nhớ, Tăng hiệu năng, Chia sẻ dữ liệu.

Phù hợp hệ thống có hàng triệu object.

Trong Order Management System, Flyweight hữu ích cho:

- Product catalog
- Discount types
- Shipping methods
- Tax categories